

PG 2nd End Laydown Tool (Simple) - User Guide

Command-line version of the PG 2nd End Laydown Tool. It automates the OrcaFlex static analysis of a **2nd-end PLET laydown** with the A&R wire: the pipeline is cut and clamped on the ramp, the PLET is welded on and lowered on the A&R wire (with buoyancy modules on a yoke or running-wire bridle), and the wire is paid out until the PLET lands and the wire is slack on the seabed.

The engine is the same code as the PyQt GUI tool. The GUI is replaced by the project JSON file (same format as the GUI "Save Current Data" file) and a small `settings.json`.

1. How the tool works

Stage	Command	What happens	Main output
Check	<code>run.bat check</code>	Reads the project JSON, checks it against the vessel database and recalculates the values the GUI used to derive	console summary
Base	<code>run.bat base</code>	Builds the base models, steps 00-03 and one step per buoyancy module	<code>Stage_1_PLET.dat</code> , <code>00..0N_LD_*.dat</code> , results Excel
Full	<code>run.bat full</code>	Same as base, then the landing loop and the wire-slackening loop	all step files, <code><job>_StaticResults.xlsx</code>
Results	<code>run.bat results</code>	Re-extracts results from the step files (after manual edits in OrcaFlex)	<code><job>_StaticResults.xlsx</code>

One run is made for each current case listed in `settings.json` (`none`, `inline`, `against`); each case has its own sub-folder.

PG 2nd End Laydown (Simple) - tool flowchart

run.bat <command> | inputs: input_data.json (GUI project format) + settings.json | outputs in analysis_folder/<current>/

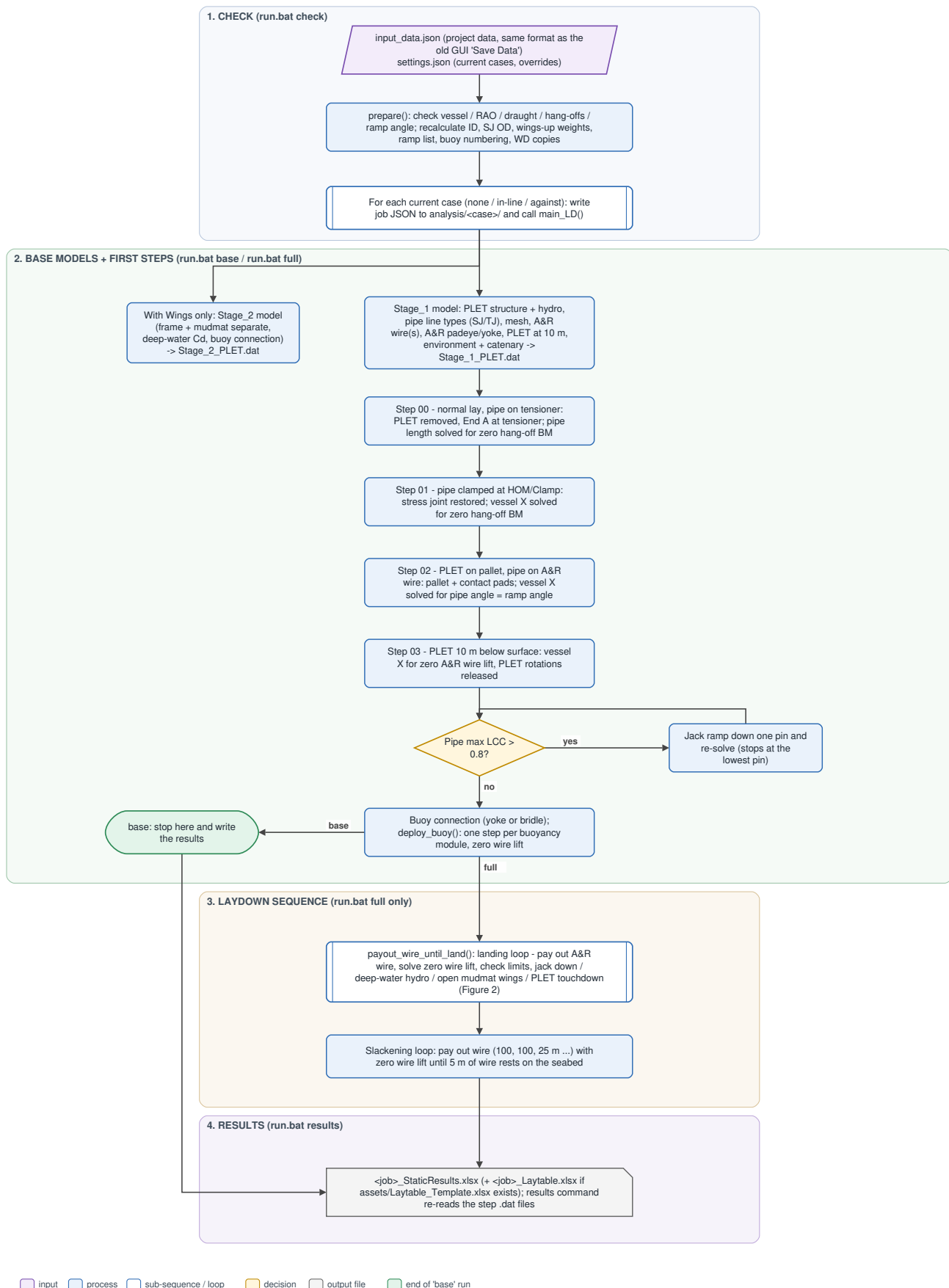


Figure 1 - Tool flowchart (also in docs/flowchart_overview.png / .svg)

2. Quick start

1. Install a **full, licensed OrcaFlex**. The Demo cannot be driven from Python (no `OrcFxAPI.dll`).
2. Run `run.bat info`. The first run creates `venv` and installs `requirements.txt`.
3. Put the project in `input_data.json`. Any JSON saved by the old GUI works as-is (`examples/Yoke.json` = yoke buoyancy connection, `examples/Bridle.json` = running-wire bridle).
4. Edit `settings.json` (analysis folder, current cases, optional overrides).
5. Run `run.bat check` and fix any reported input error.
6. Run `run.bat base`, open `Stage_1_PLET.dat` and the first steps in OrcaFlex and check them.
7. Run `run.bat full` for the complete sequence.
8. Review `<job>_StaticResults.xlsx` (and `<job>_Laytable.xlsx` when the template is installed).
9. If you adjust step files by hand, run `run.bat results` to re-extract the results.

The console shows the progress; the full solver history is in `<case>/<case><job>.log` inside each case folder.

3. Inputs

Project JSON (`input_data.json`)

Block	Content
<code>General</code>	Client / project / scope / revision, water depth <code>WD</code> , seawater density, current option and current profile
<code>pipeGeneral</code> + pipe line type block	Single pipe or pipe-in-pipe, contents, line type name; OD, WT, coating, CRA lining, E / Ramberg-Osgood, drag and added mass, seabed friction, DNV-ST-F101 / API RP 1111 factors
<code>SJGeneral</code>	Stress joint (length, OD from ID + 2 WT) and transition joint length
<code>vesselGeneral</code>	Vessel (<code>Seven Vega</code> / <code>Seven Oceans</code> / <code>Seven Navica</code>), RAO and draught, NL ramp angle, minimum / custom ramp angles, pipe hang-off, A&R mode (Single / Dual with beam, Vega only) and sheave
<code>structureGeneral</code>	PLET option (frame only / frame + mudmat without or with wings), dimensions, weights, CoG, perforation, anchor flange, A&R attachment (single padeye or A&R yoke with hinge stiffness)
<code>buoyGeneral</code>	Buoyancy on/off, connection (yoke or running wire), connection points, yoke length / weight, buoyancy module table (uplift, weight, length, sling)
<code>sequenceGeneral</code>	Pallet step (wire length, ramp angle), SJ clamp option, deployment option (fixed ramp angle or jack down at limits), maximum payout per step, VM / strain / LCC limits, wing-opening clearance, landing option

`check` recalculates: pipe ID (OD - 2 WT), DNV OD / thickness / E, stress-joint OD, wings-up weights (frame + mudmat), mudmat connection offsets, ramp object name, the ramp-angle list (vessel pins; Dual Mode ≥ 59 deg; custom angles for Vega), buoy and stiffness-row numbering, `WD_Max` from `WD`. It stops with a clear message when a vessel, RAO, hang-off, sheave or ramp angle does not exist in the vessel database.

settings.json

Key	Meaning
<code>input_file</code> , <code>analysis_folder</code>	Project JSON and job folder
<code>current_options</code>	Current cases to run: <code>none</code> , <code>inline</code> (0 deg), <code>against</code> (180 deg)
<code>orcaflex_dll</code>	Full path to <code>OrcFxAPI.dll</code> when OrcaFlex is not found automatically
<code>overrides</code>	Values merged into the project JSON before the run, e.g. <code>{"vesselGeneral": {"SelectedRampAngle": "76.1"}}</code>

4. Analysis logic

Step	Configuration	How it is solved
Base models	<code>Stage_1_PLET.dat</code> : PLET (integrated frame + mudmat with wings up), pipe with stress / transition joints, A&R wire(s), A&R padeye or yoke, catenary start shape. <code>Stage_2_PLET.dat</code> (with wings only): frame and mudmat separate, deep-water drag	Built from the vessel template
00	Normal-lay ramp angle, pipe on the tensioner (PLET and wires removed)	Pipe length for zero hang-off bending moment
01	Pipe clamped at the HOM / clamp, stress joint restored	Vessel X for zero hang-off bending moment
02	PLET on the pallet, pipe on the A&R wire (pallet step wire length)	Vessel X so the pipe angle matches the ramp angle
03	PLET 10 m below the surface	Vessel X for zero A&R wire lift, then PLET rotations released; jack down while pipe max LCC > 0.8
Buoy steps	One step per buoyancy module (yoke or bridle)	Wire length for full submergence, zero wire lift
Landing loop (full)	Wire paid out step by step to the seabed	See Figure 2
Slackening (full)	PLET on the seabed	Wire paid out until 5 m rests on the seabed

"Zero wire lift" means the vessel is moved until the A&R wire leaves the sheave at the ramp angle (within 1 deg); the same idea as the zero hang-off moment used for the pipe.

payout_wire_until_land() - landing loop (one pass = one static step)

Repeats until the PLET clearance is below 5 m; then the wire-slackening loop starts

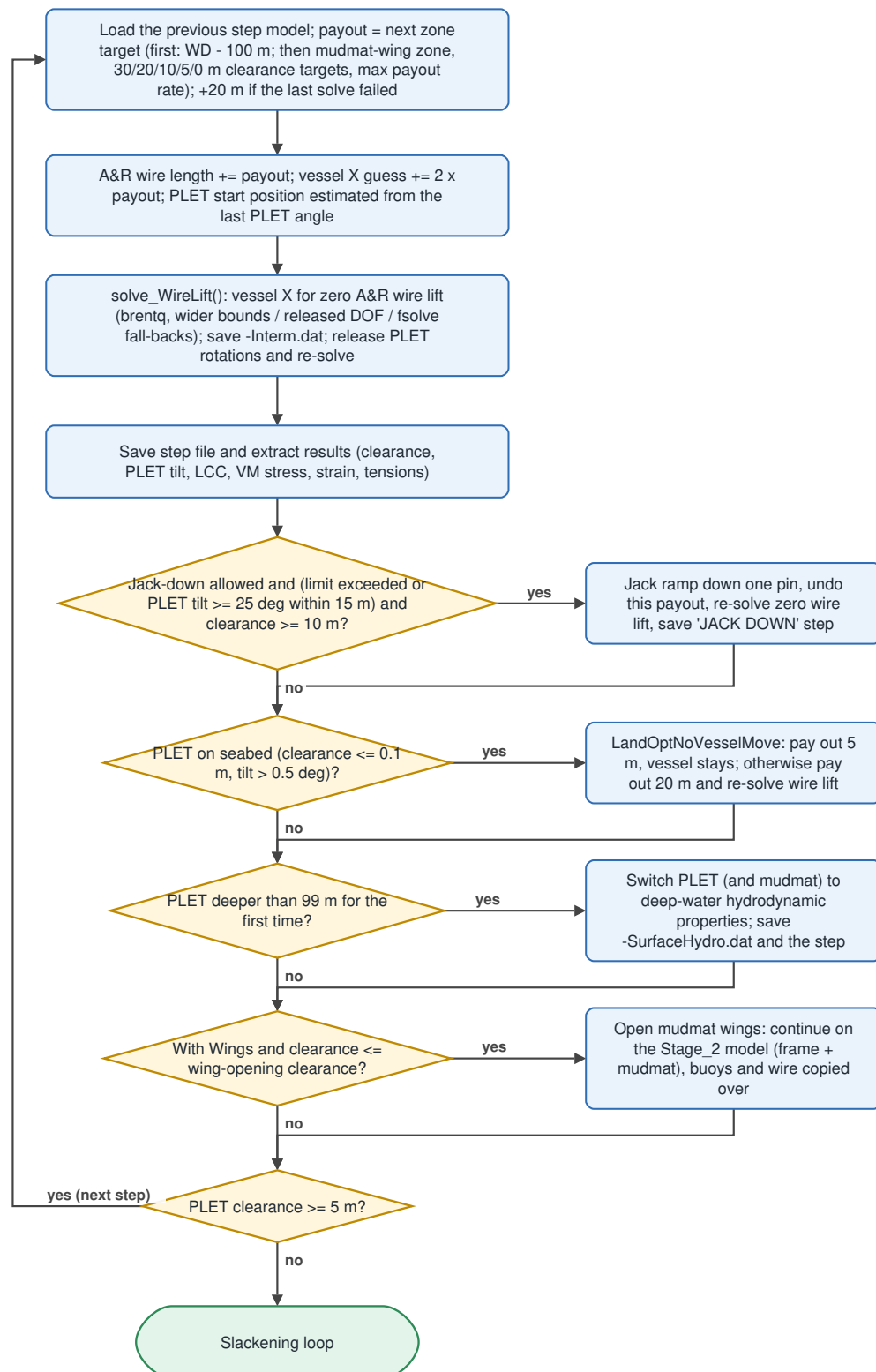


Figure 2 - Landing loop: one pass per static step

5. Outputs

All in `analysis_folder/<noCurrent | InLineCurrent | AgainstCurrent>/:`

File	Content
<code><job>.json</code>	Prepared project data used for the run
<code>Stage_1_PLET.dat, Stage_2_PLET.dat</code>	Base models
<code>NN_LD_<ramp>_Stage_<1/2>.dat</code>	Static steps (Stage 2 after the mudmat wings opened)
<code>...-Interm.dat, ...-SurfaceHydro.dat, ...-BuoyYoke.dat, ...-Failed.dat</code>	Intermediate, pre-hydro-switch, buoy-connection and failed-solve models (not used for results)
<code><case><job>.log</code>	Solver log
<code><job>_StaticResults.xlsx</code>	One row per step: ramp angle, vessel X / movement, wire payout, PLET position / clearance / tilt, pipe and wire tensions, bending moments, max stress / strain / LCC, layback, bollard pull, status and limit warnings
<code><job>_Laytable.xlsx</code>	Company lay table (only when <code>assets/Laytable_Template.xlsx</code> exists)

6. Troubleshooting

Symptom	Cause / action
Exception: OrcaFlex not found	No full OrcaFlex install; install it or set <code>orcaflex_dll</code>
INPUT ERROR: ... from check	The message names the field; use <code>run.bat info</code> for valid vessel names, RAOs, hang-offs, sheaves and pins
<code>...-Failed.dat</code> in the case folder	A zero-lift solve failed; the log shows the attempts. Open the file, adjust the start position and re-solve by hand, then <code>run.bat results</code>
"Previous step did not converge. Manual adjustment needed." in the status	Result extraction failed for that step; check it in OrcaFlex
No <code><job>_Laytable.xlsx</code>	Export the <code>Laytable_Template</code> Google Sheet to <code>assets/Laytable_Template.xlsx</code>
A & R / bollard pull warnings in the status column	Wire tension above the winch limit or bollard pull above the vessel limit (<code>assets/vessel_database/<v>/<v>_coords.json</code>)

7. Code map

File	Responsibility
<code>run.py / run.bat</code>	Command line entry point, settings, one run per current case
<code>laydown/prepare.py</code>	Checks the project JSON and recalculates GUI-derived values (no OrcaFlex needed)
<code>laydown/a_buildSequence.py</code>	<code>main_LD()</code> : base models, steps 00-03, buoy steps, laydown sequence, Excel export
<code>laydown/ b_builderFunctions.py</code>	OrcaFlex builders: line types, mesh, PLET / mudmat + hydrodynamics, A&R wire / beam / yoke, buoy yoke / bridle, pallet, catenary
<code>laydown/ c_sequenceFunctions.py</code>	Zero wire lift and pallet solvers, payout zones, jack-down, buoy deployment, landing and slackening loops
<code>laydown/ d_extractFunctions.py</code>	Static result extraction, lay table formatting, re-extraction
<code>laydown/constants.py, utils.py</code>	Constants, vessel-database path and fallbacks, helpers
<code>assets/vessel_database/</code>	Vessel templates and coordinates
<code>examples/</code>	GUI test projects (Yoke, Bridle)